Chapitre 0 — Avant tout | Partie D — Des formes | leçon 0.42 sur 153 | difficulté 1 / 10

0.42. Dans l'autre sens : sens -1

Le carré de taille 2, avec un seul changement : le sens passe de 1 à -1. La lettre tourne dans l'autre sens.

1. Ce qu'il faut comprendre

On reprend le carré de taille 2 (le 0.35), avec un seul changement : le 5e réglage, le **sens**, passe de `1` à `-1`.

Ce qui est nouveau ici : le sens. `1` tourne comme les **aiguilles d'une montre** : droite, bas, gauche, haut. `-1` tourne dans **l'autre sens** : bas, droite, haut, gauche.

Le coin de départ ne change pas : en haut à gauche. Seule la première direction change : avec `1`, la lettre part vers la **droite** ; avec `-1`, vers le **bas**.

-1 dans un octet est rangé 255 : `carre` lit tout ce qui dépasse 127 comme « l'autre sens ».

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <carre>` → 0.33.3. La fonction carre() — tourner en carré
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <carre>  // une tuile qui tourne en carré`	Demande à la console « carre » (ce que c'est : une tuile qui tourne en carré). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en carré ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// Le carré de taille 2 (le 0.35), avec un seul changement : le sens, 1 → -1.`	Un commentaire : une phrase pour celui qui lit (« Le carré de taille 2 (le 0.35), avec un seul changement : le sens, 1 → -1. »). La console l'ignore.
`//  carre(10, 8, ALPHABET[0], 2, -1, 250, 1);`	Un commentaire : une phrase pour celui qui lit (« carre(10, 8, ALPHABET[0], 2, -1, 250, 1); »). La console l'ignore.
`//  |`	Un commentaire : une phrase pour celui qui lit (« | »). La console l'ignore.

`//                                +-- sens : -1, dans l'autre sens (1 : comme les aiguilles)`	Un commentaire : une phrase pour celui qui lit (« +-- sens : -1, dans l'autre sens (1 : comme les aiguilles) »). La console l'ignore.
`// sens 1 (le 0.35) :      sens -1 (ici) :`	Un commentaire : une phrase pour celui qui lit (« sens 1 (le 0.35) : sens -1 (ici) : »). La console l'ignore.
`//   → → → → ↓           ↓ ← ← ← ←`	Un commentaire : une phrase pour celui qui lit (« → → → → ↓ ↓ ← ← ← ← »). La console l'ignore.
`//   ↑ . . . ↓           ↓ . . . ↑`	Un commentaire : une phrase pour celui qui lit (« ↑ . . . ↓ ↓ . . . ↑ »). La console l'ignore.
`//   ↑ . A . ↓           ↓ . A . ↑`	Un commentaire : une phrase pour celui qui lit (« ↑ . A . ↓ ↓ . A . ↑ »). La console l'ignore.
`//   ↑ . . . ↓           ↓ . . . ↑`	Un commentaire : une phrase pour celui qui lit (« ↑ . . . ↓ ↓ . . . ↑ »). La console l'ignore.
`//   ↑ ← ← ← ←           → → → → ↑`	Un commentaire : une phrase pour celui qui lit (« ↑ ← ← ← ← → → → → ↑ »). La console l'ignore.
`carre(10, 8, ALPHABET[0], 2, -1, 250, 1);`	Appel de la fonction carre de la console, avec les arguments 10, 8, ALPHABET[0], 2, -1, 250, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  carre()         440 o  ← main()  — une fonction fournie par le langage
  main()           38 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
```

```
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans carre()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans carre()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans carre(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  carre()   × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

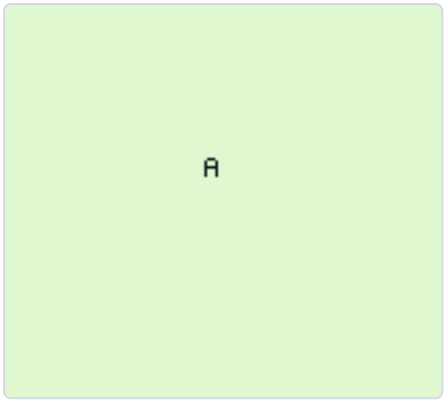
```
int main() {
  // Le carré de taille 2 (le 0.35), avec un seul changement : le sens, 1 → -1.
  //
  //   carre(10, 8, ALPHABET[0], 2, -1, 250, 1);
  //                                |
  //                                +-- sens : -1, dans l'autre sens (1 : comme les aiguilles)
  //
  // sens 1 (le 0.35) :      sens -1 (ici) :
  //
  //   → → → → ↓            ↓ ← ← ← ←
  //   ↑ . . . ↓            ↓ . . . ↑
  //   ↑ . A . ↓            ↓ . A . ↑
  //   ↑ . . . ↓            ↓ . . . ↑
  //   ↑ ← ← ← ←            → → → → ↑
  carre(10, 8, ALPHABET[0], 2, -1, 250, 1);

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

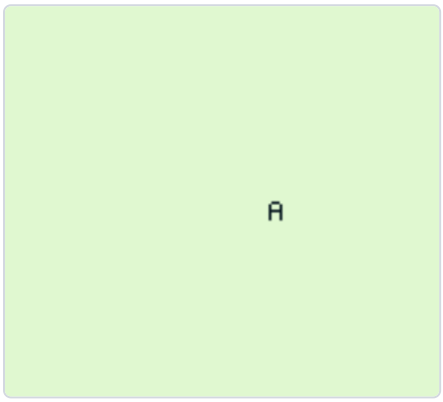
la toute première image — la console vient de démarrer — image 1



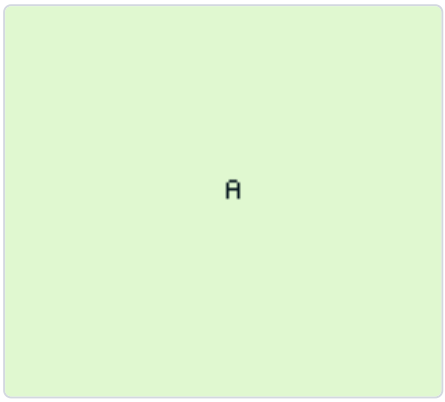
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Le carré de taille 2, mais la lettre part vers le bas et tourne dans l'autre sens.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre coins : (8, 6), (12, 6), (12, 10), (8, 10)
- ✓ jamais plus loin que la taille 2
- ✓ sens -1 : du coin, il DESCEND
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.42.1. Dans l'autre sens : sens -1 — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++